

```
Sahil = ["Sahil", 123456, 23, 5.9, 80]
print(Sahil)
print(type(Sahil))
```

```
['Sahil', 123456, 23, 5.9, 80]
<class 'list'>
```

```
Sahil = ("Sahil", 123456, 23, 5.9, 80)
print(Sahil)
print(type(Sahil))
```

```
('Sahil', 123456, 23, 5.9, 80)
<class 'tuple'>
```

```
Sahil = {"Sahil", 123456, 23, 5.9, 80}
print(Sahil)
print(type(Sahil))
```

```
{123456, 80, 5.9, 'Sahil', 23}
<class 'set'>
```

PYTHON DATA STRUCTURES

Lists, Tuples, Sets

In real-world programming, we often need to store multiple values inside a single variable.

Imagine:

- 100 student names
- 1000 product prices
- 10,000 sensor readings

Creating separate variables is impossible.

That is why Python provides **Data Structures**.

They allow us to:

- Store multiple values
- Organize data
- Perform operations easily

▼ LIST

A List is:

- Ordered
- Changeable (Mutable)
- Allows duplicates
- Indexed

Lists are created using square brackets []

Used when:

- Data changes frequently
- Order matters
- Duplicates are allowed

Real-life examples:

- Student names
- Marks
- Shopping cart items

```
# Creating a list
students = ["Rahul", "Aman", "Neha", "Priya"]

print(students)
# a = "Sahil"
# print(a[0:4])
# Accessing elements (Indexing)
print(students[0]) # First element
print(students[-1]) # Last element

# Slicing
print(students[1:3])
```

```
['Rahul', 'Aman', 'Neha', 'Priya']
Rahul
Priya
['Aman', 'Neha']
```

```
numbers = [10, 20, 30]
print(numbers)
# Modify element
numbers[0] = 100
# numbers[3] = 100
print(numbers)

# Add element
numbers.append("Sahil")
print(numbers)

# Remove element
numbers.remove("Sahil")
print(numbers)
```

```
[10, 20, 30]
[100, 20, 30]
[100, 20, 30, 'Sahil']
[100, 20, 30]
```

```
A = [67,67,68,56,45]
A = [67,67,68,56,45]
A = [67,67,68,56,45]
A = [67,67,68,56,45]
A = [67,67,68,56,45]
```

```
marks = [85, 90, 78, 92, 48]
V = "65"
print("V")
average = sum(marks) / len(marks)
c = sum(marks)
print(c)
a = str((c / 500) * 100)
a = (c / 500) * 100
print(type(a))
# a = (sum(marks) / 500) * 100
b = "%"

# print(a + b)

print("Average Marks:", average)
print("marks")
```

```
V
393
<class 'float'>
Average Marks: 78.6
marks
```

✓ IN-DEPTH UNDERSTANDING OF LIST IN PYTHON

A List is:

- Ordered collection
- Mutable (changeable)
- Allows duplicates
- Indexed
- Can store multiple data types

Lists are the most used data structure in Python.

Used in:

- Data Science
- Machine Learning
- Web Development
- Competitive Programming

If you understand Lists deeply, you understand 60% of practical Python.

```
# Creating a list
numbers = [10, 20, 30, 40, 50]

print(numbers)
print(type(numbers))
```

```
[10, 20, 30, 40, 50]
<class 'list'>
```

✓ LIST PROPERTIES

1. Ordered → Maintains insertion order
2. Mutable → Can change elements
3. Allows duplicates
4. Indexed → Access via position
5. Can store mixed data types

```
mixed_list = ["Rahul", 21, 85.5, True]

print(mixed_list)
```

```
['Rahul', 21, 85.5, True]
```

```
students = ["Rahul", "Aman", "Neha", "Priya"]

# Positive indexing
print(students[0])
print(students[2])

# Negative indexing
print(students[-1])
print(students[-2])
```

```
Rahul
Neha
Priya
Neha
```

```
numbers = [10, 20, 30, 40, 50, 60]

print(numbers[1:4]) # 20, 30, 40
print(numbers[:3]) # first 3 elements
print(numbers[3:]) # from index 3 onward
print(numbers[::-2]) # step slicing
```

```
[20, 30, 40]
[10, 20, 30]
[40, 50, 60]
[10, 30, 50]
```

✓ LIST IS MUTABLE (VERY IMPORTANT)

Mutable means: You can modify, update, remove, and add elements.

This makes List powerful and flexible.

```
marks = [80, 85, 90]

# Modify element
marks[0] = 100
print(marks)

# Add element
marks.append(95)
print(marks)

# Insert at specific position
marks.insert(1, 88)
print(marks)

# Remove element
marks.remove(85)
print(marks)

# Remove using index
marks.pop()
print(marks)
```

```
[100, 85, 90]
[100, 85, 90, 95]
[100, 88, 85, 90, 95]
[100, 88, 90, 95]
[100, 88, 90]
```

✓ IMPORTANT LIST METHODS

- `append()` → Add at end
- `insert()` → Add at specific index
- `remove()` → Remove by value
- `pop()` → Remove by index
- `sort()` → Sort list
- `reverse()` → Reverse list
- `len()` → Length of list

```
numbers = [5, 2.98, 9, 1, 7]

numbers.sort()
print("Sorted:", numbers)

numbers.reverse()
print("Reversed:", numbers)

print("Length:", len(numbers))
```

```
Sorted: [1, 2.98, 5, 7, 9]
Reversed: [9, 7, 5, 2.98, 1]
Length: 5
```

✓ REAL LIFE EXAMPLE — STUDENT MARK ANALYSIS

Problem: You have marks of students. Find:

- Highest marks
- Lowest marks
- Average

```
marks = [85, 90, 78, 92, 88]

print("Highest:", max(marks))
print("Lowest:", min(marks))
print("Average:", sum(marks) / len(marks))
```

```
Highest: 92
Lowest: 78
Average: 86.6
```

✓ DUPLICATES IN LIST

Lists allow duplicates.

This is useful when:

- Recording transactions
- Storing scores
- Logging repeated data

```
scores = [85, 90, 85, 92, 90]

print(scores)
```

```
[85, 90, 85, 92, 90]
```

✓ NESTED LISTS (2D LIST)

List inside list.

Used for:

- Matrix
- Table data
- CSV rows

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
print(matrix[2][1])
a = [1, 2, 3]
# a[1][3]
a = [1, 2, 3]
b = [1, 2, 3]
c = [1, 2, 3]
mat = [a, b, c]
class_8 = [[12,13,14,15],[23,24,25,26],[19,17,13,15]]
print(mat)
print(mat[0])
# print(matrix[1][2])
```

```
3
[[1, 2, 3], [1, 2, 3], [1, 2, 3]]
[1, 2, 3]
```

```
mat = ['v', 'f', 'k']
print(mat)
print(mat[0])
```

```
['v', 'f', 'k']
v
```

✓ WHEN TO USE LIST?

Use List when:

- Data changes frequently
- Order matters
- Duplicates allowed
- You need indexing

Lists are dynamic and flexible.

```
# Mini challenge:
# Take 5 numbers from user and store in list.
# Print max, min, and average.

nums = []

nums.append(int(input("Enter number 1: ")))
nums.append(int(input("Enter number 2: ")))
nums.append(int(input("Enter number 3: ")))
nums.append(int(input("Enter number 4: ")))
nums.append(int(input("Enter number 5: ")))

print("Maximum:", max(nums))
print("Minimum:", min(nums))
print("Average:", sum(nums)/len(nums))
```

```
Enter number 1: 34
Enter number 2: 23
Enter number 3: 23
Enter number 4: 23
Enter number 5: 23
Maximum: 34
Minimum: 23
Average: 25.2
```

✓ LIST PRACTICE QUESTIONS (VERY BASIC)

Answer in a new code cell.

1. Create a list named numbers containing: 10, 20, 30, 40.
2. Print the entire list.
3. Print the first element of the list.
4. Print the last element of the list using negative indexing.
5. Find and print the length of the list.
6. Add the number 50 to the end of the list.
7. Insert the number 15 at index position 1.
8. Change the value 30 to 300 in the list.
9. Remove the value 20 from the list.
10. Remove the last element using pop().
11. Create a list of 5 city names and print the third city.
12. Create a list with duplicate values (example: 1, 2, 2, 3, 3). Print the list.
13. Sort the list [5, 2, 9, 1] in ascending order.
14. Reverse the list [1, 2, 3, 4].
15. Create a list of marks (80, 85, 90, 95) and print:
 - Maximum marks
 - Minimum marks
 - Average marks

```
# =====
# SOLUTIONS - LIST BASIC PRACTICE
# =====

# 1
numbers = [10, 20, 30, 40]

# 2
print("Full list:", numbers)

# 3
print("First element:", numbers[0])

# 4
print("Last element:", numbers[-1])

# 5
print("Length of list:", len(numbers))

# 6
numbers.append(50)
print("After append 50:", numbers)

# 7
numbers.insert(1, 15)
print("After insert 15 at index 1:", numbers)

# 8
numbers[numbers.index(30)] = 300
print("After changing 30 to 300:", numbers)

# 9
numbers.remove(20)
print("After removing 20:", numbers)

# 10
numbers.pop()
print("After pop():", numbers)
```